

```

from google.colab import drive
drive.mount('/content/drive')

zip_file_path = "/content/drive/MyDrive/Datasets/Copy of devnagari digit.zip"
extract_path = "/content/"

# Unzip the file silently (-q) to the specified extraction path (-d)
!unzip -q "{zip_file_path}" -d "{extract_path}"

print(f"Successfully extracted '{zip_file_path}' to '{extract_path}'")

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
 Successfully extracted '/content/drive/MyDrive/Datasets/Copy of devnagari digit.zip' to '/content/'.

```

from google.colab import drive
drive.mount('/content/drive')

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```

import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist

(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train, x_test = x_train / 255.0, x_test / 255.0
print(x_train.shape)

x_train = x_train.reshape(-1, 28 * 28)
x_test = x_test.reshape(-1, 28 * 28)

y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)

print(y_test)

```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 ————— 0s 0us/step
 (60000, 28, 28)
 [[0. 0. 0. ... 1. 0. 0.]
 [0. 0. 1. ... 0. 0. 0.]
 [0. 1. 0. ... 0. 0. 0.]
 ...

```
[0. 0. 0. ... 0. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]
[0. 0. 0. ... 0. 0. 0.]]
```

```
import tensorflow as tf
from tensorflow import keras
num_classes = 10
input_shape = (28*28, )
model = keras.Sequential(
    [
        keras.layers.Input(shape=input_shape),
        keras.layers.Flatten(),
        keras.layers.Dense(64, activation="sigmoid"),
        keras.layers.Dense(128, activation="sigmoid"),
        keras.layers.Dense(256, activation="sigmoid"),
        keras.layers.Dense(num_classes, activation="softmax"),
    ]
)
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 64)	50,240
dense_1 (Dense)	(None, 128)	8,320
dense_2 (Dense)	(None, 256)	33,024
dense_3 (Dense)	(None, 10)	2,570

Total params: 94,154 (367.79 KB)

Trainable params: 94,154 (367.79 KB)

Non-trainable params: 0 (0.00 KB)

```
model.compile(
    optimizer="sgd",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

```

batch_size = 128
epochs = 20

callbacks = [
    keras.callbacks.ModelCheckpoint(filepath="model_at_epoch_{epoch}.keras"),
    keras.callbacks.EarlyStopping(monitor="val_loss", patience=4 ),
]

history = model.fit(
    x_train,
    y_train,
    batch_size=batch_size,
    epochs=epochs,
    validation_split=0.2,
    callbacks=callbacks,
)

```

```

Epoch 1/20
375/375 ————— 3s 6ms/step - accuracy: 0.1103 - loss: 2.3067 - val_accuracy: 0.1060 - val_loss: 2.3017
Epoch 2/20
375/375 ————— 2s 5ms/step - accuracy: 0.1133 - loss: 2.3012 - val_accuracy: 0.1060 - val_loss: 2.3010
Epoch 3/20
375/375 ————— 3s 8ms/step - accuracy: 0.1151 - loss: 2.2999 - val_accuracy: 0.1060 - val_loss: 2.3003
Epoch 4/20
375/375 ————— 3s 7ms/step - accuracy: 0.1198 - loss: 2.2985 - val_accuracy: 0.1060 - val_loss: 2.2995
Epoch 5/20
375/375 ————— 2s 5ms/step - accuracy: 0.1186 - loss: 2.2973 - val_accuracy: 0.1060 - val_loss: 2.2970
Epoch 6/20
375/375 ————— 2s 5ms/step - accuracy: 0.1206 - loss: 2.2961 - val_accuracy: 0.1060 - val_loss: 2.2954
Epoch 7/20
375/375 ————— 2s 5ms/step - accuracy: 0.1271 - loss: 2.2942 - val_accuracy: 0.1035 - val_loss: 2.2934
Epoch 8/20
375/375 ————— 2s 5ms/step - accuracy: 0.1253 - loss: 2.2927 - val_accuracy: 0.1060 - val_loss: 2.2923
Epoch 9/20
375/375 ————— 2s 7ms/step - accuracy: 0.1334 - loss: 2.2909 - val_accuracy: 0.1060 - val_loss: 2.2901
Epoch 10/20
375/375 ————— 3s 8ms/step - accuracy: 0.1419 - loss: 2.2886 - val_accuracy: 0.2008 - val_loss: 2.2893
Epoch 11/20
375/375 ————— 2s 6ms/step - accuracy: 0.1424 - loss: 2.2864 - val_accuracy: 0.1135 - val_loss: 2.2867
Epoch 12/20
375/375 ————— 2s 5ms/step - accuracy: 0.1456 - loss: 2.2836 - val_accuracy: 0.1995 - val_loss: 2.2819
Epoch 13/20
375/375 ————— 2s 5ms/step - accuracy: 0.1704 - loss: 2.2802 - val_accuracy: 0.2222 - val_loss: 2.2787
Epoch 14/20
375/375 ————— 2s 5ms/step - accuracy: 0.1737 - loss: 2.2765 - val_accuracy: 0.1060 - val_loss: 2.2751
Epoch 15/20
375/375 ————— 2s 5ms/step - accuracy: 0.1773 - loss: 2.2721 - val_accuracy: 0.1954 - val_loss: 2.2705
Epoch 16/20
375/375 ————— 3s 7ms/step - accuracy: 0.2058 - loss: 2.2667 - val_accuracy: 0.2306 - val_loss: 2.2644

```

```
Epoch 17/20
375/375 ————— 3s 8ms/step - accuracy: 0.2254 - loss: 2.2601 - val_accuracy: 0.3390 - val_loss: 2.2565
Epoch 18/20
375/375 ————— 2s 5ms/step - accuracy: 0.2479 - loss: 2.2520 - val_accuracy: 0.2861 - val_loss: 2.2466
Epoch 19/20
375/375 ————— 2s 5ms/step - accuracy: 0.2724 - loss: 2.2415 - val_accuracy: 0.2954 - val_loss: 2.2350
Epoch 20/20
375/375 ————— 2s 5ms/step - accuracy: 0.2953 - loss: 2.2282 - val_accuracy: 0.4876 - val_loss: 2.2199
```

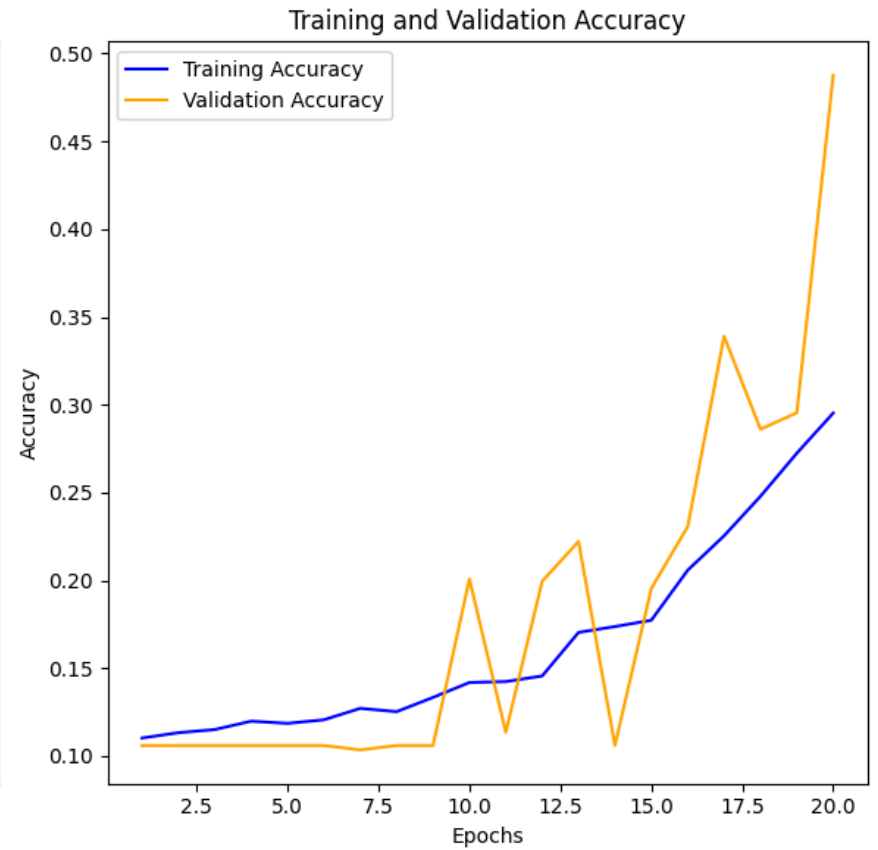
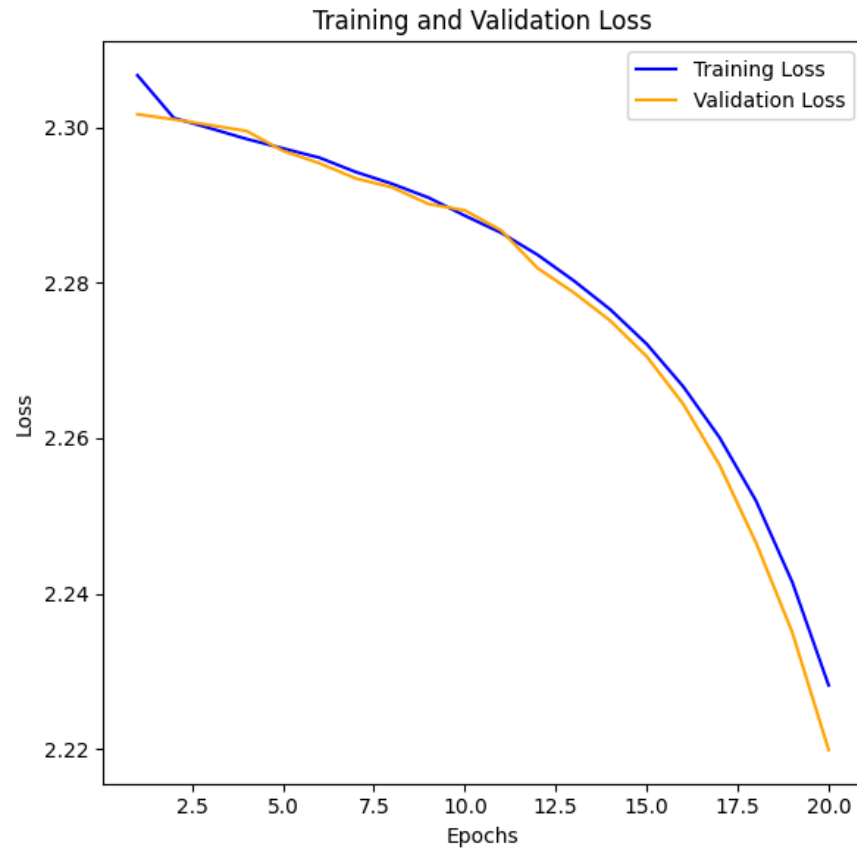
```
import matplotlib.pyplot as plt

train_loss = history.history['loss']
val_loss = history.history['val_loss']

train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.plot(range(1, len(train_loss) + 1), train_loss, label='Training Loss', color='blue')
plt.plot(range(1, len(val_loss) + 1), val_loss, label='Validation Loss', color='orange')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training and Validation Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(range(1, len(train_acc) + 1), train_acc, label='Training Accuracy', color='blue')
plt.plot(range(1, len(val_acc) + 1), val_acc, label='Validation Accuracy', color='orange')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Training and Validation Accuracy')
plt.legend()
plt.tight_layout()
plt.show()
```



```
import os
import numpy as np
from PIL import Image
```

```
def one_hot_encode(labels, num_classes):
    return np.eye(num_classes)[labels]
```

```
def load_dataset(folder_path, img_size=(28, 28)):
    images = []
    labels = []
    class_names = []

    # Sort classes to keep label order consistent
    classes = sorted(os.listdir(folder_path))
```

```

for label, class_name in enumerate(classes):
    class_path = os.path.join(folder_path, class_name)

    if os.path.isdir(class_path):
        class_names.append(class_name)

    for file in os.listdir(class_path):
        img_path = os.path.join(class_path, file)

        try:
            # Load image using PIL
            img = Image.open(img_path).convert("RGB")

            # Resize image
            img = img.resize(img_size)

            # Convert to numpy & normalize (0-1)
            img_array = np.array(img) / 255.0

            images.append(img_array)
            labels.append(label)

        except:
            continue # skip bad files

return np.array(images), np.array(labels), class_names

```

```
train_path = "/content/DevanagariHandwrittenDigitDataset/Train"
```

```
X_train, y_train, class_names = load_dataset(train_path)
```

```
# One-hot encode labels
```

```
y_train_encoded = one_hot_encode(y_train, len(class_names))
```

```
print("X_train shape:", X_train.shape)
```

```
print("y_train shape:", y_train_encoded.shape)
```

```
print("Classes:", class_names)
```

```
X_train shape: (17000, 28, 28, 3)
```

```
y_train shape: (17000, 10)
```

```
Classes: ['digit_0', 'digit_1', 'digit_2', 'digit_3', 'digit_4', 'digit_5', 'digit_6', 'digit_7', 'digit_8', 'digit_9']
```

```
test_path = "/content/DevanagariHandwrittenDigitDataset/Train"
```

```
X_test, y_test, _ = load_dataset(test_path)
```

```
y_test_encoded = one_hot_encode(y_test, len(class_names))

print("X_test shape:", X_test.shape)
print("y_test shape:", y_test_encoded.shape)
```

```
X_test shape: (17000, 28, 28, 3)
y_test shape: (17000, 10)
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
```

```
model = Sequential([
    Flatten(input_shape=(28, 28, 3)),

    Dense(64, activation='sigmoid'),
    Dense(128, activation='sigmoid'),
    Dense(256, activation='sigmoid'),

    Dense(10, activation='softmax')
])
```

```
model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_dim`
super().__init__(**kwargs)
```

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 2352)	0
dense_4 (Dense)	(None, 64)	150,592
dense_5 (Dense)	(None, 128)	8,320
dense_6 (Dense)	(None, 256)	33,024
dense_7 (Dense)	(None, 10)	2,570

```
Total params: 194,506 (759.79 KB)
```

```
Trainable params: 194,506 (759.79 KB)
```

```
Non-trainable params: 0 (0.00 B)
```

```
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy', # because of one-hot encoding
```

```
metrics=['accuracy']
)
```

```
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
```

```
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)
```

```
checkpoint = ModelCheckpoint(
    "best_model.h5",
    monitor='val_loss',
    save_best_only=True
)
```

```
history = model.fit(
    X_train,
    y_train_encoded,
    batch_size=128,
    epochs=20,
    validation_split=0.2,
    callbacks=[early_stop, checkpoint]
)
```

Epoch 1/20

103/107 ————— 0s 12ms/step - accuracy: 0.3540 - loss: 1.9073WARNING:absl:You are saving your model as an HDF5 file via `

107/107 ————— 3s 19ms/step - accuracy: 0.5788 - loss: 1.4673 - val_accuracy: 0.0000e+00 - val_loss: 7.7999

Epoch 2/20

107/107 ————— 2s 15ms/step - accuracy: 0.8916 - loss: 0.4106 - val_accuracy: 0.0000e+00 - val_loss: 9.1408

Epoch 3/20

107/107 ————— 1s 14ms/step - accuracy: 0.9508 - loss: 0.1954 - val_accuracy: 0.0000e+00 - val_loss: 9.7366

Epoch 4/20

107/107 ————— 1s 9ms/step - accuracy: 0.9666 - loss: 0.1288 - val_accuracy: 0.0000e+00 - val_loss: 10.0831

```
test_loss, test_accuracy = model.evaluate(X_test, y_test_encoded)
```

```
print("Test Loss:", test_loss)
```

```
print("Test Accuracy:", test_accuracy)
```

532/532 ————— 1s 2ms/step - accuracy: 0.6537 - loss: 2.0909

Test Loss: 2.090860366821289

Test Accuracy: 0.653705894947052


```
model.save("finalmodel.h5")
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is cons

```
from tensorflow.keras.models import load_model
```

```
loaded_model = load_model("finalmodel.h5")
```

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until you

```
loss, acc = loaded_model.evaluate(X_test, y_test_encoded)
```

```
print("Loaded Model Accuracy:", acc)
```

532/532 ————— 2s 2ms/step - accuracy: 0.6537 - loss: 2.0909
Loaded Model Accuracy: 0.653705894947052

```
predictions = loaded_model.predict(X_test)
```

532/532 ————— 1s 2ms/step

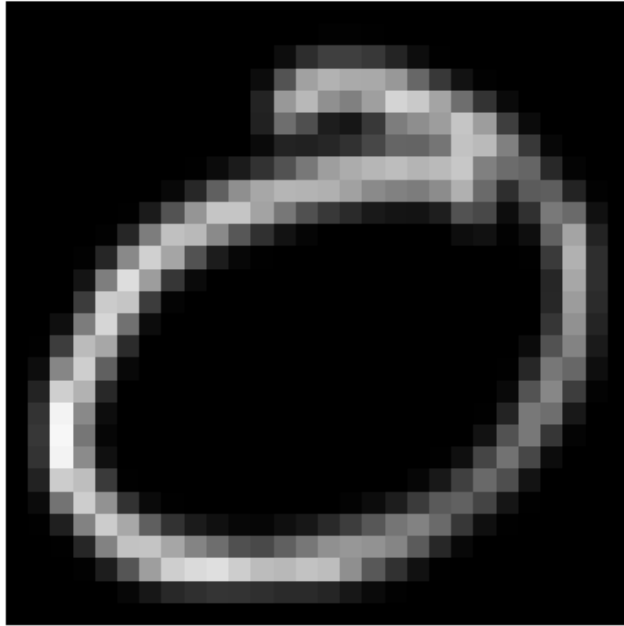
```
predicted_labels = np.argmax(predictions, axis=1)
```

```
true_labels = np.argmax(y_test_encoded, axis=1)
```

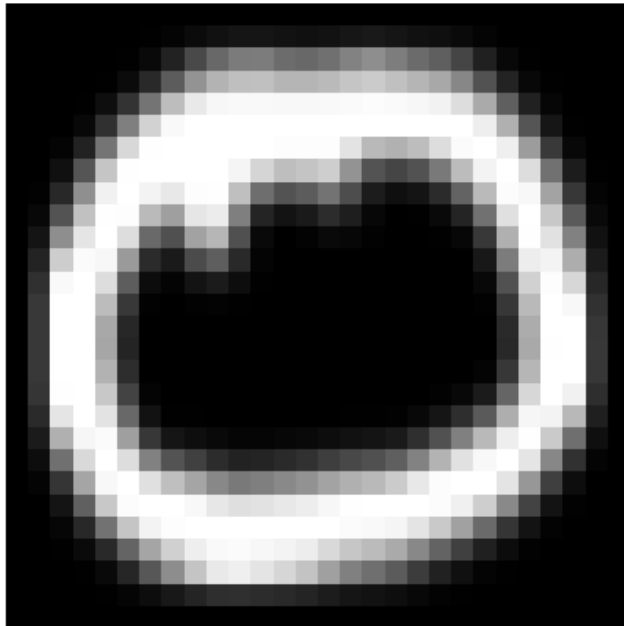
```
import matplotlib.pyplot as plt
```

```
for i in range(5):  
    plt.imshow(X_test[i])  
    plt.title(f"Pred: {predicted_labels[i]}, Actual: {true_labels[i]}")  
    plt.axis("off")  
    plt.show()
```


Pred: 0, Actual: 0



Pred: 0, Actual: 0



Pred: 0, Actual: 0

